

Оптимизация потребления токенов в интеллектуальных агентных системах на базе больших языковых моделей

Авторы: Круглов Б. Е., Девятерикова М. В.

Организация: Омский государственный технический университет (ОмГТУ)

Омск, 2026

Проблема масштабирования *LLM*-агентов

Переход к агентам архитектуры *AutoGPT* требует передачи всей истории на каждом шаге. Алгоритмическая сложность затрат достигает критического предела $O(N^2)$.

Базовая математическая модель стоимости сессии:

$$C_{total} = \sum_{i=1}^N (c_{in} \cdot T_{in}^{(i)} + c_{out} \cdot T_{out}^{(i)})$$

Переполнение окна контекста провоцирует эффект *lost in the middle*. Модель теряет фокус на центре объемных инструкций. Это фатально для многошаговых задач.

Строгая логика вытесняется галлюцинациями.

Цель исследования

Фундаментальная задача — радикальное снижение стоимости вызовов *API*. Требуется минимизация объема входных данных T_{in} до жесткой константы. Логика рассуждений модели обязана оставаться неизменной. Экономическая целесообразность продукта зависит от преодоления квадратичного барьера потребления токенов. Необходимо полностью исключить искажение фактов.

Алгоритм гибридной памяти

Векторный поиск страдает «семантической близорукостью» и теряет хронологию. Базовый отбор контекста d для запроса q выполняется через косинусное сходство:

$$\text{sim}(q, d) = \frac{q \cdot d}{||q|| \cdot ||d||}$$

Гибридная функция R решает проблему хронологии. Фактор забывания λ обеспечивает вытеснение старых фактов более актуальными событиями:

$$R = \alpha \cdot \text{sim}(v_q, v_j) + (1 - \alpha) \cdot e^{-\lambda(t_{\text{now}} - t_j)}$$

В контекст передаются исключительно фрагменты, преодолевшие жесткий порог отсечения ($R > \theta$, где $\theta = 0.5$).

Иерархическое кэширование контекста

Системный *prompt* разбивается на строгие уровни. Слой *L1* содержит спецификации инструментов и *JSON*-схемы. Он аппаратно кэшируется провайдером. Слой *L3* отвечает за текущий запрос и генерируется с нуля. Стоимость сессии учитывает тариф записи c_w и сниженный тариф чтения c_γ :

$$C_{cache} = c_w \cdot T_{sys} + \sum_{i=1}^N (c_\gamma \cdot T_{sys} + c_{in} \cdot T_{user}^{(i)} + c_{out} \cdot T_{out}^{(i)})$$

Тариф чтения из кэша составляет лишь десятую часть от стандартного ввода ($c_\gamma \approx 0.1 \cdot c_{in}$). Вычислительная нагрузка на *GPU* радикально снижается.

Адаптивная маршрутизация запросов

Модель *Llama-3-8B* выступает первичным диспетчером задач. Архитектура включает три независимых узла. Модуль *Intent Classifier* определяет тип запроса. Узел *Complexity Scorer* оценивает количество логических шагов. Модуль *Cost Optimizer* принимает решение на основе бюджета. Тривиальные операции мгновенно отсекаются от флагманских *LLM*. Тяжелые запросы направляются старшим моделям.

Результаты экспериментального исследования

Оценка проводилась на выборке из 5 сценариев длительностью до 50 шагов. В задачах поиска данных экономия токенов достигала 70.4%. Общий объем контекста на 50-м шаге упал с 45000 до 4000 токенов. Деградация точности до 86.4% прямо коррелирует с агрессивной работой семантического фильтра при $\theta = 0.5$.

Метрика	<i>Baseline</i>	<i>Optimized</i>	Эффект
Входные токены	100%	34.7%	−65.3%
Задержка (<i>Latency</i>)	7.1 сек	4.6 сек	−35.0%

Практическая значимость

Точность выполнения задач сохранилась вопреки жесткому сжатию контекста. Операционные расходы упали ровно в 2.8 раза. Квадратичный барьер алгоритмической сложности $O(N^2)$ успешно преодолен. Предложенная методология формирует надежный архитектурный базис. Мощные академические *LLM*-инструменты готовы к массовому внедрению в ресурсограниченные мобильные приложения.